# Multisets of $O(k)$-invariants under a labelling symmetry group

Working notes

October 14, 2026

## 1 Setup

Let $L = (v_1, v_2, \ldots, v_n)$ be an ordered list of $n$ vectors, each $v_i \in \mathbb{F}^k$, where $\mathbb{F}$ is usually $\mathbb{R}$ but may be $\mathbb{C}$.

### 1.1 Invariants

**Definition 1** (Dots)**.** For $1 \le i \le j \le n$ let

$$d_{ij} = v_i \cdot v_j = v_i^{\mathsf{T}} v_j.$$

There are $n + \binom{n}{2}$ of these (self-pairs included). The product is bilinear (no complex conjugation) also when $\mathbb{F} = \mathbb{C}$.

**Definition 2** (Epsilons)**.** For $1 \le i_1 < i_2 < \cdots < i_k \le n$ let

$$e_{i_1 i_2 \cdots i_k} = \mathrm{Det}\big[v_{i_1}, v_{i_2}, \ldots, v_{i_k}\big].$$

There are $\binom{n}{k}$ of these. The inequalities are strict: any repeated index gives a zero determinant.

For non-canonically-ordered indices we use $d_{ji} = d_{ij}$ and $e_{\ldots j \ldots i \ldots} = -e_{\ldots i \ldots j \ldots}$ (total antisymmetry), so any index list can be brought to canonical (increasing) order, gaining a sign $\mathrm{sgn}(\pi)$ for the sorting permutation $\pi$ in the case of an $e$.

### 1.2 The vector $w$

Let $w$ be the column vector containing every $d$ and every $e$ in a fixed, predetermined order. Any order is allowed, but it is convenient to keep together the entries that form a single orbit under the permutation part of the group $G$ (below).

### 1.3 The symmetry group $G$

$G$ is generated by

- a subgroup of $S_n$ acting on the labels $1, \ldots, n$, typically a product $S(n_1) \times S(n_2) \times \cdots$ permuting labels only within blocks. For example, for $n = 5$, $S(3) \times S(2)$ permutes $\{v_1, v_2, v_3\}$ among themselves and $\{v_4, v_5\}$ among themselves;
- optionally, a parity factor $P = \{1, p\}$, so that $|G \times P| = 2|G|$.

**Definition 3** (Parity)**.** $p$ acts on every vector by the same fixed reflection,

$$p : v_i \mapsto R\, v_i, \qquad R = \mathrm{diag}(1, \ldots, 1, -1),$$

that is, it flips the sign of the last component of every vector.

**Lemma 1.** *For every $k$, $p$ leaves each $d$ unchanged and negates each $e$:*

$$p : d_{ij} \mapsto d_{ij}, \qquad p : e_{i_1 \cdots i_k} \mapsto -e_{i_1 \cdots i_k}.$$

*Proof.* $R^{\mathsf{T}}R = I$, so $(Rv_i)^{\mathsf{T}}(Rv_j) = v_i^{\mathsf{T}}v_j$. Also $\mathrm{Det}[Rv_{i_1}, \ldots, Rv_{i_k}] = \det(R)\,\mathrm{Det}[v_{i_1}, \ldots, v_{i_k}]$ and $\det R = -1$. □

*Remark* 1. The proof uses only $R^{\mathsf{T}}R = I$ and $\det R = -1$. So any reflection, or more generally any orthogonal matrix with $\det = -1$, acts identically on $w$, and the choice of which component to flip is immaterial. Because $R$ is real, the $d$'s are invariant under either a bilinear or a Hermitian product. The $e$'s, however, pair naturally with the bilinear $d$.

*Remark* 2 (Point inversion is *not* the right parity). The alternative $v \mapsto -v$ (that is, $R = -I_k$) has $\det(-I_k) = (-1)^k$. It negates the $e$'s only for odd $k$. For even $k$ (for example $k = 2$, where $-I_2$ is a rotation by $\pi$) it leaves $w$ completely unchanged. This is why parity is defined as a reflection above.

### 1.4 Group action on $w$ and the multiset

For $g \in G$, the vector $g \cdot w$ is obtained as follows:

1. relabel the indices of each entry according to the permutation part of $g$;
2. restore canonical index order, using $d_{ji} = d_{ij}$ and the antisymmetry sign for each $e$;
3. if $g$ contains $p$, negate every $e$ (Lemma 1).

The resulting entries are re-expressed in terms of the original (canonical) $d$'s and $e$'s, keeping the slot positions of $w$ fixed. Define the multiset

$$\mathrm{MS} = \{\, g \cdot w \mid g \in G \,\},$$

with multiplicities, so that $|\mathrm{MS}| = |G|$ counted with multiplicity.

## 2 Worked example: $k = 2$, $L = (a, b, p)$

Here $n = 3$. To avoid a clash with the parity element, the vector named $p$ in $L$ is written as a subscript only. There are $3 + 3 = 6$ dots and $\binom{3}{2} = 3$ epsilons. Ordered by orbits of $S(2) \times S(1)$ (bars separate the orbits):

$$w = \big(\, d_{aa}, d_{bb} \;\big|\; d_{pp} \;\big|\; d_{ab} \;\big|\; d_{ap}, d_{bp} \;\big|\; e_{ab} \;\big|\; e_{ap}, e_{bp} \,\big)^{\mathsf{T}}.$$

Let $\sigma = (ab)$. Under $\sigma$: $d_{ab} \mapsto d_{ba} = d_{ab}$, $e_{ab} \mapsto e_{ba} = -e_{ab}$, and $e_{ap} \leftrightarrow e_{bp}$ with no sign change.

### 2.1 $G = S(2) \times S(1)$

$$\mathrm{MS} = \left\{ \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ e_{ab} \\ e_{ap} \\ e_{bp} \end{pmatrix}}_{1}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ -e_{ab} \\ e_{bp} \\ e_{ap} \end{pmatrix}}_{\sigma} \right\}$$

### 2.2 $G = S(2) \times S(1) \times P$

$$\mathrm{MS} = \left\{ \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ e_{ab} \\ e_{ap} \\ e_{bp} \end{pmatrix}}_{1}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ -e_{ab} \\ e_{bp} \\ e_{ap} \end{pmatrix}}_{\sigma}, \underbrace{\begin{pmatrix} d_{aa} \\ d_{bb} \\ d_{pp} \\ d_{ab} \\ d_{ap} \\ d_{bp} \\ -e_{ab} \\ -e_{ap} \\ -e_{bp} \end{pmatrix}}_{p}, \underbrace{\begin{pmatrix} d_{bb} \\ d_{aa} \\ d_{pp} \\ d_{ab} \\ d_{bp} \\ d_{ap} \\ e_{ab} \\ -e_{bp} \\ -e_{ap} \end{pmatrix}}_{\sigma p} \right\}$$

In $\sigma p$, the swap sign and the parity sign cancel on $e_{ab}$.

## 3 Layer 2: embedding the multiset

The multiset MS is not yet a usable output. Next we apply a continuous, injective and algebraic *multiset embedding*

$$\Phi : \big(\mathbb{F}^m\big)^{|G|}/S_{|G|} \longrightarrow \mathbb{F}^D, \qquad m = \dim w,$$

and express each component of $\Phi(\mathrm{MS})$ explicitly as a polynomial in the $d$'s and $e$'s. Several choices of $\Phi$ are possible. We start with the Bursarial embedding because its outputs are explicit low-degree polynomials, which makes it the easiest to work with.

### 3.1 The Bursarial embedding

Index the slots of $w$ from 0 to $m-1$, so $w = (w_0, \ldots, w_{m-1})^{\mathsf{T}}$. For each $g \in G$ define

$$q_g(x) = \sum_{j=0}^{m-1} (g \cdot w)_j \, x^j,$$

and multiply one linear factor in $y$ for each group element:

$$B(x, y) = \prod_{g \in G} \big(y + q_g(x)\big).$$

**Elementary symmetric polynomials.** For $N$ quantities $z_1, \ldots, z_N$, the *c-th elementary symmetric polynomial* is the sum of all products of $c$ of them with distinct indices:

$$E_c(z_1, \ldots, z_N) = \sum_{1 \le i_1 < i_2 < \cdots < i_c \le N} z_{i_1} z_{i_2} \cdots z_{i_c}, \qquad E_0 = 1.$$

For example, $E_1 = z_1 + \cdots + z_N$ and $E_N = z_1 z_2 \cdots z_N$. When $N = 3$, $E_2 = z_1 z_2 + z_1 z_3 + z_2 z_3$. They are the coefficients that appear when a product of linear factors is multiplied out:

$$\prod_{i=1}^{N} (y + z_i) = \sum_{c=0}^{N} y^{N-c} \, E_c(z_1, \ldots, z_N).$$

(For $N = 2$ this is $(y + z_1)(y + z_2) = y^2 + y(z_1 + z_2) + z_1 z_2$.)

**Applying this to $B$.** Take $N = |G|$ and let the $z$'s be the polynomials $q_g(x)$, one for each $g \in G$. Write

$$E_c(x) \equiv E_c\big(\{q_g(x)\}_{g\in G}\big) = \sum_{\substack{\{g_1,\dots,g_c\}\subseteq G \\ (c \text{ distinct elements})}} q_{g_1}(x)\, q_{g_2}(x) \cdots q_{g_c}(x).$$

Then

$$B(x,y) = \sum_{c=0}^{|G|} y^{|G|-c}\, E_c(x).$$

Each $q_g$ has degree (at most) $m-1$ in $x$, so $E_c(x)$, a sum of products of $c$ of them, is a polynomial in $x$ of degree (at most) $c(m-1)$. Write its coefficients as $E_{c,j}$:

$$E_c(x) = \sum_{j=0}^{c(m-1)} E_{c,j}\, x^j.$$

(The notation $[x^j]\, f$ is sometimes used for the coefficient of $x^j$ in $f$, so that $E_{c,j} = [x^j]\, E_c$.) The leading term $y^{|G|}$ ($c = 0$) carries no information. The embedding is the list of all the remaining coefficients:

$$\Phi(\mathrm{MS}) = \big(E_{c,j}\big)_{1\le c\le |G|,\ 0\le j\le c(m-1)},$$

which has

$$D = \sum_{c=1}^{|G|} \big(c(m-1)+1\big) = |G| + (m-1)\,\frac{|G|(|G|+1)}{2}$$

components. This is the construction used in Echinocoder's `Cinf_sympy_bursar_embedder_for_array_of_reals_as_multiset.py`, with $n \to |G|$ and $k \to m$.

**Properties.**

- *Injective on multisets.* $\mathbb{F}[x][y]$ is a unique factorisation domain, and each factor $y + q_g(x)$ is monic in $y$. So $B$ determines the multiset $\{q_g\}$, and hence MS.
- *Invariant.* $B$ is a product over all of $G$, so $\Phi(\mathrm{MS})$ is unchanged when $L$ is acted on by any $g \in G$.
- *Polynomial.* $E_{c,j}$ is a homogeneous polynomial of degree $c$ in the entries of $w$, that is, in the $d$'s and $e$'s:

$$E_{c,j} = \sum_{\{g_1,\dots,g_c\}\subseteq G} \ \sum_{j_1+\cdots+j_c=j} \ \prod_{r=1}^{c} (g_r \cdot w)_{j_r}.$$

- *Order-dependent coordinates.* The slot order of $w$ decides which entries multiply which powers of $x$. Changing it changes the coordinates of $\Phi$, but not its injectivity.

**What injectivity is actually required.** We never want to recover $L$ itself. The purpose of MS is to forget exactly the information that $G$ moves. Using $d$'s and $e$'s erases the orientation of $L$ in $\mathbb{F}^k$, that is, the action of $SO(k)$. Over $\mathbb{R}$ the Gram matrix $(d_{ij})$ determines $L$ up to $O(k)$. The $e$'s then fix the handedness whenever the $v_i$ span $\mathbb{F}^k$; when they do not, there is no handedness to fix. Each factor of $G$ erases something further. For example, $S(3)$ erases which of $a, b, c$ is which, and $P$, if present, erases handedness, so that $L$ is then known only up to $O(k)$. In that case the $e$'s carry nothing beyond the $d$'s. Every $G$-invariant is then even in the $e$'s, and by Cauchy–Binet $e_I\, e_J = \det\big[d_{i_r j_s}\big]_{r,s=1}^{k}$. They are kept in $w$ here only for uniformity. A practical implementation would leave them out whenever $P \subseteq G$, because they greatly enlarge

MS and everything that then has to be pruned. (Over $\mathbb{C}$ with a bilinear product the degenerate cases, such as nonzero $v$ with $v \cdot v = 0$, still need checking.) Two notions of injectivity should therefore be kept apart:

(A) *Formal injectivity.* Treat the $m$ entries of $w$ as independent variables, and require $\Phi$ to separate $\{g \cdot w\}$ for *every* $w \in \mathbb{F}^m$. This is all the Bursarial embedding knows about.

(B) *Injectivity on the multisets that actually occur.* Require

$$\Phi\big(\mathrm{MS}(L)\big) = \Phi\big(\mathrm{MS}(L')\big) \implies \mathrm{MS}(L) = \mathrm{MS}(L'),$$

equivalently $L' = g \cdot (h\, L)$ for some $g \in G$ and orthogonal $h$ (special orthogonal if $P \not\subseteq G$). Only those $w = w(L)$ lying on the variety cut out by the Gram and Plücker relations need to be separated.

(B) is the target. (A) implies (B), but (B) asks less of $\Phi$, so it should be achievable with fewer outputs.

**Redundancy.** $\Phi$ is deliberately oversized, for two separate reasons. First, it does not know that MS is structured. MS is the $G$-orbit of a single point $w$, and each slot of $g \cdot w$ is $\pm$ an entry from the same $G$-orbit block of $w$, so many coefficients vanish identically and many others are functionally dependent on the rest. Second, it does not know the Gram and Plücker relations among the $d$'s and $e$'s, which are what separate target (B) from target (A). Exploiting both kinds of redundancy is the next layer. The present goal is only the oversized expressions, which are certainly injective, generated algorithmically and verifiably for general $G$.

### 3.2 Example: $k = 2$, $L = (a, b, p)$, $G = S(2) \times S(1)$

With the slot order of Section 2, $m = 9$, $|G| = 2$ and $D = 2 + 8 \cdot 3 = 26$:

$$q_1(x) = d_{aa} + d_{bb}x + d_{pp}x^2 + d_{ab}x^3 + d_{ap}x^4 + d_{bp}x^5 + e_{ab}x^6 + e_{ap}x^7 + e_{bp}x^8,$$
$$q_\sigma(x) = d_{bb} + d_{aa}x + d_{pp}x^2 + d_{ab}x^3 + d_{bp}x^4 + d_{ap}x^5 - e_{ab}x^6 + e_{bp}x^7 + e_{ap}x^8.$$

The $c = 1$ block ($E_1 = q_1 + q_\sigma$, 9 coefficients $E_{1,0}, \ldots, E_{1,8}$) is

$$\big(\, d_{aa}{+}d_{bb},\ d_{aa}{+}d_{bb},\ 2d_{pp},\ 2d_{ab},\ d_{ap}{+}d_{bp},\ d_{ap}{+}d_{bp},\ 0,\ e_{ap}{+}e_{bp},\ e_{ap}{+}e_{bp} \,\big),$$

which already shows a vanishing coefficient and repeated ones. The $c = 2$ block ($E_2 = q_1 q_\sigma$, 17 coefficients $E_{2,0}, \ldots, E_{2,16}$) contains, for example,

$$E_{2,12} = 2\big(d_{ap}e_{ap} + d_{bp}e_{bp}\big) - e_{ab}^2.$$

This is where the information about $e_{ab}$ lost from $E_1$ reappears.

## 4 Layer 3: removing redundancy

There is no obviously best method for removing the redundancy in $\Phi$. This section records what we are optimising and which reductions are safe.

### 4.1 Design objectives

1. *Injectivity on every input.* Target (B) must hold for every $L$, including degenerate ones: coincident or parallel vectors, and $v_i$ that fail to span $\mathbb{F}^k$. Separation for generic $L$ alone is not acceptable. We expect sets that separate everywhere to be strictly larger than sets that separate only generic $L$. The extra outputs are needed only to resolve confusions on degenerate inputs.

2. *Few outputs.* The number of outputs is to be minimised subject to (1). It can never be smaller than the dimension of the quotient, which for $n \geq k$ is $nk - \frac{1}{2}k(k-1)$. By Dym and Gortler, about twice the input dimension suffices if dense combinations are allowed, but see (3).

3. *Low complexity of each output, as a function of the raw inputs.* Separating invariants that have high degree and depend on delicate cancellations can be so "spiky" that, plotted against the components of the $v_i$, they look almost like white noise. Anything that later *looks at* the embedding then cannot generalise from it. So the reduction in (2) is balanced against keeping each output simple. Specifically:

   - Complexity is measured by *degree in the components of the* $v_i$, not by the size of the expression in $d$'s and $e$'s. Each $d$ has degree 2 and each $e$ has degree $k$. A finer measure is the multidegree, with one degree for each vector $v_i$. Every Gram, Plücker and mixed $d$–$e$ relation is homogeneous, so rewriting through them changes the expression but never the function, and never its degree. The growth in expression size from rewriting $e\,e\,e\,e$ as $d$'s is therefore a problem of how outputs are displayed, not of how smooth they are.
   - Degree does control smoothness. By a Bernstein-type inequality, a polynomial of degree $r$ restricted to the unit sphere has angular gradient at most $r$ times its maximum modulus there.
   - Outputs should be *homogeneous* in $L$, or better still multi-homogeneous, so that each one scales cleanly with the overall size of $L$. Splitting an output into its homogeneous parts never harms injectivity, because the parts determine the whole, although it does increase the count. The Bursarial $E_{c,j}$ are homogeneous in $L$ only when $k = 2$.
   - A homogeneous output $f$ of degree $r$ can optionally be replaced by $\operatorname{sign}(f)\,|f|^{1/r}$. This is a bijection of $\mathbb{R}$, so injectivity is preserved, and every output then scales linearly with $L$. The cost is that the output is no longer differentiable where $f = 0$.
   - Dense combinations that mix many monomials or degrees, such as generic linear projections, are disfavoured.

4. *Evaluation cost* is a secondary concern and is expected to be modest.

### 4.2 Reductions that are always safe

Removing an output that is given by an *explicit, verifiable formula* (here, a linear one) in terms of the outputs kept can never harm injectivity, and the formula itself is the proof. The following reductions therefore need no judgement:

- if $P \subseteq G$, omit the $e$'s from $w$ from the outset;
- drop coefficients that vanish identically;
- drop exact duplicates and scalar multiples;
- drop any output in the linear span of those kept. This can be detected formally (in the $d$'s and $e$'s) or as functions of $L$, where the Gram, Plücker and mixed relations hold automatically.

If the reduced set still separates, then every discarded output is still *determined* by the outputs kept. Equal kept outputs imply equal MS, so a discarded output is a well-defined function of the kept ones. What changes beyond the reductions above is that this function is generally not linear, and not even polynomial. For example, $x$ is determined by $x^3$ on $\mathbb{R}$, but only through a cube root. Discarded invariants typically satisfy polynomial equations over the kept ones, and

injectivity is what selects the correct root. The safety of such a reduction therefore cannot be read off from an identity among the outputs. It needs a certificate valid on every input: either an exact algebraic computation (of the *separating variety*), or an explicit decoding algorithm, with case splits for degenerate inputs, that is proved to recover the orbit of $L$ as in target (B). Tests at random points, such as Jacobian rank or counting preimages, can only propose candidates or reject them.

### 4.3 Levels of certification

"Certified" means that there is evidence that an output set separates orbits on *every* input (target (B)). That evidence comes in levels of decreasing strength.

**Level 0: by construction.** Injectivity follows from a general theorem, and nothing needs to be checked for the particular $(n, k, G)$. Example: the unpruned Bursarial embedding (by unique factorisation).

**Level 1: explicit identity.** A Level 0 set from which outputs have been removed, each removed output being an explicit formula in the kept ones ("0", "= output 7", "$= \frac{1}{2}\,\text{output}_3 - \text{output}_9$"). The code verifies each formula exactly, in rational arithmetic, and injectivity is inherited from Level 0. These are the reductions of Section 4.2.

**Level 2: proof of separation.** The removed outputs are not formulas of the kept ones, but there is a proof that the kept set separates every input. The proof can take one of three forms:

(a) a theorem proved for a whole family of cases and recorded in these notes, so that the code need only check that the requested $(n, k, G)$ satisfies its hypotheses;

(b) an exact computer-algebra computation for this particular $(n, k, G)$, such as showing that the separating variety is exactly the union of the graphs of the group action. The result is stored as an artefact that can be checked again;

(c) a decoder that reconstructs the orbit of $L$ from the outputs, with a written proof, including case splits for degenerate inputs, that it succeeds on every input. The code implements the decoder, and tests exercise it.

Forms (a) and (c) rest on proofs checked by people, and (b) rests on the correctness of the computer-algebra system. None is formally verified. "Certified" therefore means that a proof exists that we stand behind, not that the proof has been checked by machine.

**Level 3: evidence only.** Tests at random points pass (full Jacobian rank, correct preimage counts, no confusable pairs found by search), but there is no proof. Generic injectivity is then plausible, but nothing reliable is known about degenerate inputs. Level 3 sets are *uncertified*.

## 5 Code interface (draft for discussion)

This section specifies what code in Echinocoder should do, before any code is written. It is intended to be iterated on.

### 5.1 Two phases: build once, evaluate many times

**Build (once, may be slow).** The user supplies a specification and receives an embedding function together with its metadata:

```
embed_fn, meta = build_invariant_embedding(k, group_spec, options)
```

- `k`: the dimension of each vector.
- `group_spec`: defines $n$, the labels, and $G$. $G$ is a direct product of symmetric groups, one for each *species*, for example $((a, b), (p))$ for $S(2) \times S(1)$, together with a flag for whether $P$ is included. This matches symatom's existing `VectorType`/`Context`, which already builds $w$ (as `repS`) and the signed-permutation action, and should be reused. Parity is not yet a group element in symatom. This class of groups is sufficient for now. Arbitrary permutation subgroups are out of scope, but the design should not rule them out.
- `options`: control the construction (see below).

Symbolic work happens only at build time: constructing $w$ and $G$, forming the embedding, applying reductions, and recording provenance.

**Evaluate (many times, must be cheap).** The target framework is PyTorch, as used by the IWPC package (`https://github.com/jjhwilkinson/iwpc`, which depends on `torch` and `lightning`). `embed_fn` is therefore a `torch.nn.Module`:

```
Y = embed_fn(X)      # X.shape == (..., n, k)  ->  Y.shape == (..., D)
```

- Any leading batch dimensions ("`...`") are allowed, and may differ from call to call. $n$ and $k$ are fixed at build time, and inputs with any other trailing shape are rejected.
- Along the $n$ axis, `X[..., i, :]` is the vector with the $i$-th label of `group_spec`. That ordering convention is part of the specification.
- Internally the pipeline is: `X` $\rightarrow$ the $d$'s and $e$'s ($w$, shape $(\ldots, m)$) $\rightarrow$ the polynomials chosen at build time $\rightarrow$ optional root transform $\rightarrow$ `Y`. Only tensor operations are used, so autograd works throughout. No group elements are enumerated at evaluation time. Any constants are registered as buffers, so that `.to(device)` and `state_dict()` behave as usual.
- `Y` must separate orbits for *every* input, and must be differentiable almost everywhere, so that it can be used for backprop. Differentiability may fail at corners: for example, the root transform is not differentiable where $f = 0$. Its gradient there must be defined explicitly (finite, rather than `inf` or `nan`).
- *IWPC compatibility, without depending on IWPC.* Echinocoder must not have a hard dependency on IWPC. It should, however, fit naturally into IWPC, using the same packages (`torch` and `numpy`) and the same conventions as IWPC's `Encoding` base class:
  - an `nn.Module` whose `input_shape` and `output_shape` are registered as integer buffers, and which acts on tensors of shape `(..., *input_shape)`;
  - `input_shape` $= (n, k)$ and `output_shape` $= D$;
  - a flattened variant with `input_shape` $= nk$ (vector-major: component $c$ of $v_i$ at position $ik + c$), because IWPC's `&` concatenation needs vector inputs;
  - NumPy-style docstrings, as in IWPC.

  Wrapping the module as an IWPC `Encoding` should then be a matter of a few lines. Echinocoder may ship such an adaptor, provided IWPC is imported only when the adaptor is used, so that nothing else in Echinocoder needs IWPC installed.
- NumPy inputs may be accepted for convenience, for example in tests and documentation, by conversion to tensors.

### 5.2 Options (initial list)

- `embedder`: which multiset embedding to use (layer 2). Initially only `"bursar"`, but chosen from a registry so that others can be added.
- `reductions`: which reductions to apply, as an ordered list or a level. Early versions may support only `none`. Next would come the safe reductions of Section 4 (zero, duplicate, scalar multiple, linear span, which are Level 1; and dropping the $e$'s when $P \subseteq G$, which is Level 0 for the smaller $w$, because the $d$'s alone determine $L$ up to $O(k)$). Level 2 reductions are applied only if the corresponding proof, artefact or decoder exists in the code. The main builder never returns a Level 3 set. See Section 4.3.
- `homogenise`: split outputs into homogeneous (or multi-homogeneous) parts in $L$.
- `root_normalise`: apply $\operatorname{sign}(f)|f|^{1/r}$ to each homogeneous output of degree $r$.
- `field`: $\mathbb{R}$ initially. Whether to support $\mathbb{C}$ is still open.
- `dtype`: the precision of evaluation, `float32` by default or `float64`. It affects only the evaluator. The build-time symbolic work is exact, with integer or rational coefficients. In keeping with the usual `torch` conventions, the module can also be converted afterwards with `.to(dtype)`.

An embedding that works in `float64` but not in `float32` is already too fragile to be useful. Keeping outputs of low degree and low noise (objective 3 of Section 4) is partly what makes `float32` sufficient, so the designs should aim to work in `float32`. As a diagnostic, the build could compare `float32` and `float64` evaluations on random inputs and record the discrepancy for each output in `meta`.

### 5.3 Metadata

`meta` is for documentation and auditing only. It is never used in evaluation. It should be serialisable (for example as JSON), and should contain or allow generation of:

- the spec and options it was built from, plus the Echinocoder version or commit, so that the embedding can be rebuilt and checked;
- $D$, together with the slot order of $w$;
- for each output: its symbolic formula (see below), its degree and multidegree in $L$, whether it is homogeneous, any transform applied, and its provenance (for example, "$E_{2,12}$ of the Bursarial embedding");
- a log of the reductions: each discarded output, the reason, and the evidence. Examples are "identically zero", "equal to output $i$", "$= \sum_i \lambda_i$ (output $i$)", or a named certificate;
- the *certification level* (Section 4.3) and what it rests on: the theorem, the identities, the stored artefact, or the decoder together with its proof;
- a LaTeX table generator for the outputs.

**Symbolic form of the outputs.** Evaluation works from the components of $L$, but the symbolic description never expands below the $d$'s and $e$'s. These are its leaves. Nobody reading `meta` needs to see $EF - p_x q_x - p_y q_y - p_z q_z$ in place of a dot product. Degrees in $L$ are still reported, counting 2 for each $d$ and $k$ for each $e$.

The description may also use *nested definitions*, naming intermediate quantities once and referring to them by name. For example,

$$\text{enc}[23] = f\big(h(w)\big) + g\big(h(w)\big), \qquad \text{with } h(w) = \dots$$

Natural intermediates are $q_g(x)$, the elementary symmetric polynomials $E_c$ and their coefficients $E_{c,j}$, and orbit sums. This applies to both the machine-readable form (a set of named definitions, such as a DAG of expressions) and the LaTeX output (a table of outputs plus a table of definitions). Its purpose is to keep each displayed formula short. Whether to use nesting, and how deeply, is a presentation choice. It does not affect evaluation.

### 5.4 Placement and reuse

The code lives in a new Echinocoder subpackage next to symatom and symcoder. It should not inherit from the existing `MultisetEmbedder`, which works on one input at a time and has no group specification.

symatom already builds something close to $w$ (`repS`) and the signed-permutation action of $G$ on it, and it should be reused where it matches. It may do exactly what is needed here, or something subtly different. Before any part of it is relied on, its behaviour must be checked against the definitions of Section 1, in particular:

- which atoms appear: all self-dots, cross-dots and $e$'s, and nothing else;
- the canonical index order, and the sign conventions for $e$'s under relabelling;
- parity. symatom's `Atom.sign` is known to conflate the sign from permuting arguments with the sign from spatial parity. This was harmless while there was no parity operation, but $P$ is now being added.

Tests should compare the $w$ and $g \cdot w$ obtained from symatom with an independent, direct implementation of Section 1 on small cases. symcoder's encoders are numerical, mostly based on sorting, and are not Bursarial. So the main thing to reuse there is perhaps its design experience rather than its code.

### 5.5 Scale

The initial goal is $k \le 5$ and $n \le 10$. Larger would be welcome. The goal has strong consequences. For a single species with $n = 10$, $k = 5$:

$$m = \tbinom{10}{2} + 10 + \tbinom{10}{5} = 307, \qquad |G| = 10! \approx 3.6 \times 10^6 \ (\times 2 \text{ with } P), \qquad D_{\text{Bursar}} \approx 2 \times 10^{15},$$

whereas the quotient dimension, which is the lower bound on the number of outputs, is only $nk - \frac{1}{2}k(k-1) = 40$. Hence:

- The unpruned Bursarial embedding is feasible only for small $|G|$. There it serves as a reference baseline, against which reductions can be validated.
- For large $G$, the build cannot enumerate $G$, and cannot first form the unpruned output and then prune it. It must construct the reduced outputs directly. One way is to work with orbits of monomials in $w$ (orbit sums), described by the pattern of their labels rather than by listing group elements. symatom's orbit enumerators may help here.
- Evaluation should use tensor contractions, for example $\sum_{i,j} d_{ij}^2 = \text{tr}(\mathbf{d}^2)$, rather than expanded lists of terms, whose length grows with orbit size.

Even $k \le 5$, $n \le 10$ may prove too big. It is a goal, not a requirement.

### 5.6 Uncertified (Level 3) candidates

The main builder returns only sets at Levels 0–2. Level 3 candidates are useful for research, for example to see how small output sets can get and to suggest which sets are worth proving. They are available only through a separate, explicitly named research entry point, *not* through a flag or option on the main builder, so that they cannot be picked up by accident. Their `meta` states that they are uncertified (Level 3), and lists the evidence collected.